Structured Perceptron for Sequence Labeling

POS Tagging and Named Entity Recognition

CS 115B – Spring 2026

James Pustejovsky

Today's Plan

Goal: Deep dive into structured perceptron with concrete examples

Part I: Quick Review (10 minutes)

- Algorithm recap
- Key components

Part II: POS Tagging in Detail (35 minutes)

- Task definition and examples
- Feature engineering walkthrough
- Training examples with actual updates
- Viterbi decoding step-by-step

Part III: Named Entity Recognition (35 minutes)

- NER task and BIO tagging
- Feature engineering for NER
- Handling entity boundaries
- Complete training example

Structured Perceptron: Quick Recap

Goal: Learn to predict structured outputs (sequences)

Key idea:

- Score entire sequences: $\text{score}(\mathbf{x}, \mathbf{y}) = \mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y})$
- Find best sequence: $\hat{\mathbf{y}} = \arg\max_{\mathbf{y}'} \mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}')$ using Viterbi
- Update on mistakes: $\mathbf{w} \leftarrow \mathbf{w} + \Phi(\mathbf{x}, \mathbf{y}) - \Phi(\mathbf{x}, \hat{\mathbf{y}})$

Three components we'll explore in detail:

1. **Features** $\Phi(\mathbf{x}, \mathbf{y})$: How to represent sequence + labels?
2. **Inference**: How to find best sequence efficiently?
3. **Learning**: How do updates change the model?

Feature Function Decomposition

For sequences, features decompose over positions:

$$\Phi(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \mathbf{f}(y_{i-1}, y_i, \mathbf{x}, i)$$

Each local feature vector $\mathbf{f}(y_{i-1}, y_i, \mathbf{x}, i)$ can look at:

- Current word x_i and surrounding words
- Previous tag y_{i-1} and current tag y_i
- Position i
- Global properties of $\mathbf{x}$

Score decomposes:

$$\text{score}(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{n} \mathbf{w}^\top \mathbf{f}(y_{i-1}, y_i, \mathbf{x}, i)$$

This decomposition enables efficient Viterbi decoding.

Part-of-Speech Tagging

Task: Assign grammatical category to each word

Example:

The	cat	sat	on	the	mat
DET	NOUN	VERB	ADP	DET	NOUN

Penn Treebank tagset: 45 tags

- DET (determiner), NOUN, VERB, ADJ (adjective), ADV (adverb)
- ADP (adposition/preposition), PRON (pronoun), NUM (numeral)
- PROPN (proper noun), etc.

Why challenging?

- Ambiguity: "book" can be NOUN or VERB
- Context matters: "to run" vs "the run"
- Unknown words at test time

Running Example for POS Tagging

Training sentence:

The	dog	runs	fast
DET	NOUN	VERB	ADV

We'll walk through:

1. What features fire for this sentence
2. How to score different tag sequences
3. What happens during a training update
4. How to decode a new sentence

Simplified tagset for clarity:

- DET, NOUN, VERB, ADJ, ADV, ADP

Feature Templates for POS Tagging

Emission features (word-tag):

- $\langle y_i, x_i \rangle$: current tag and word
- $\langle y_i, \text{lowercase}(x_i) \rangle$: tag and lowercased word
- $\langle y_i, \text{prefix}(x_i, k) \rangle$: tag and k-char prefix
- $\langle y_i, \text{suffix}(x_i, k) \rangle$: tag and k-char suffix
- $\langle y_i, \text{contains-hyphen}(x_i) \rangle$
- $\langle y_i, \text{is-capitalized}(x_i) \rangle$

Transition features:

- $\langle y_{i-1}, y_i \rangle$: tag bigram

Context features:

- $\langle y_i, x_{i-1} \rangle$, $\langle y_i, x_{i+1} \rangle$: neighboring words

Features for "The dog runs fast"

Position 1: "The" / DET

- $\langle \text{START}, \text{DET} \rangle$ (transition from start)
- $\langle \text{DET}, \text{The} \rangle$ (emission)
- $\langle \text{DET}, \text{the} \rangle$ (lowercase)
- $\langle \text{DET}, \text{T} \rangle$ (1-char prefix)
- $\langle \text{DET}, \text{e} \rangle$ (1-char suffix)
- $\langle \text{DET}, \text{capitalized} \rangle$
- $\langle \text{DET}, \text{next=dog} \rangle$ (right context)

Each feature is a dimension in the feature vector. If it fires, value $= 1$, else 0.

In practice: hundreds of thousands to millions of features!

Features for "The dog runs fast" (continued)

Position 2: "dog" / NOUN

- $\langle \text{DET}, \text{NOUN} \rangle$ (transition)
- $\langle \text{NOUN}, \text{dog} \rangle$ (emission)
- $\langle \text{NOUN}, \text{dog} \rangle$ (lowercase, same here)
- $\langle \text{NOUN}, \text{og} \rangle$ (2-char suffix)
- $\langle \text{NOUN}, \text{prev=The} \rangle$
- $\langle \text{NOUN}, \text{next=runs} \rangle$

Position 3: "runs" / VERB

- $\langle \text{NOUN}, \text{VERB} \rangle$ (transition)
- $\langle \text{VERB}, \text{runs} \rangle$ (emission)
- $\langle \text{VERB}, \text{s} \rangle$ (suffix)
- ...

Complete Feature Vector

For the sentence-tag pair:

$$\texttt{The/DET dog/NOUN runs/VERB fast/ADV}$$

$\Phi(\mathbf{x}, \mathbf{y})$ is a sparse vector with entries like:

$$\begin{aligned}
\Phi[\langle \text{START}, \text{DET} \rangle] &= 1 \\
\Phi[\langle \text{DET}, \text{the} \rangle] &= 1 \\
\Phi[\langle \text{DET}, \text{NOUN} \rangle] &= 1 \\
\Phi[\langle \text{NOUN}, \text{dog} \rangle] &= 1 \\
\Phi[\langle \text{NOUN}, \text{VERB} \rangle] &= 1 \\
&\vdots
\end{aligned}$$

Most entries are 0 (sparse representation).

Score: $\mathbf{w}^\top \Phi(\mathbf{x}, \mathbf{y}) = \sum_{\text{active features}} w_f$

Initial Model: Random Weights

Iteration 1, Example 1: Process "The dog runs fast"

True tags: DET NOUN VERB ADV

Current weights: $\mathbf{w} = \mathbf{0}$ (initialization)

Step 1: Viterbi decoding
With $\mathbf{w} = \mathbf{0}$, all sequences have score 0!

Viterbi will return an arbitrary sequence (tie-breaking). Let's say it predicts:

Predicted: DET VERB NOUN ADV

(Wrong at positions 2 and 3!)

Step 2: Compare with gold tags $\rightarrow$ Mismatch! $\rightarrow$ Update weights

Training Update Example

Gold: DET NOUN VERB ADV
Predicted: DET VERB NOUN ADV
Update rule:

$$\mathbf{w} \leftarrow \mathbf{w} + \Phi(\mathbf{x}, \mathbf{y}_{\text{gold}}) - \Phi(\mathbf{x}, \mathbf{y}_{\text{pred}})$$

What changes?
Increase weight for features in gold:

- $w[\langle \text{DET}, \text{NOUN} \rangle] \leftarrow w[\cdot] + 1$
- $w[\langle \text{NOUN}, \text{dog} \rangle] \leftarrow w[\cdot] + 1$
- $w[\langle \text{NOUN}, \text{VERB} \rangle] \leftarrow w[\cdot] + 1$
- $w[\langle \text{VERB}, \text{runs} \rangle] \leftarrow w[\cdot] + 1$

Decrease weight for features in prediction:

- $w[\langle \text{DET}, \text{VERB} \rangle] \leftarrow w[\cdot] - 1$
- $w[\langle \text{VERB}, \text{dog} \rangle] \leftarrow w[\cdot] - 1$
- $w[\langle \text{VERB}, \text{NOUN} \rangle] \leftarrow w[\cdot] - 1$
- $w[\langle \text{NOUN}, \text{runs} \rangle] \leftarrow w[\cdot] - 1$

After First Update

Key weights changed:

Feature	Weight
$\langle \text{DET}, \text{NOUN} \rangle$	+1
$\langle \text{DET}, \text{VERB} \rangle$	-1
$\langle \text{NOUN}, \text{dog} \rangle$	+1
$\langle \text{VERB}, \text{dog} \rangle$	-1
$\langle \text{NOUN}, \text{VERB} \rangle$	+1
$\langle \text{VERB}, \text{NOUN} \rangle$	-1
$\langle \text{VERB}, \text{runs} \rangle$	+1
$\langle \text{NOUN}, \text{runs} \rangle$	-1

Effect:

- "dog" as NOUN now scores better than "dog" as VERB (+2 difference)
- DET $\rightarrow$ NOUN transition preferred over DET $\rightarrow$ VERB (+2 difference)

Processing Next Example

Training example 2:

A	fast	dog	runs
DET	ADJ	NOUN	VERB

Current model has learned:

- DET $\rightarrow$ NOUN is good $(+1)$
- NOUN $\rightarrow$ VERB is good $(+1)$

Viterbi prediction: Might predict `DET NOUN NOUN VERB`
(Gets "fast" wrong, tagging as NOUN instead of ADJ)

Update:

- Increase $w[\langle \text{DET}, \text{ADJ} \rangle]$, $w[\langle \text{ADJ}, \text{fast} \rangle]$
- Decrease $w[\langle \text{DET}, \text{NOUN} \rangle]$ (but it was $+1$, now becomes 0)

Viterbi Decoding: Setup

Task: Find highest-scoring sequence

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}'} \text{score}(\mathbf{x}, \mathbf{y}') = \arg\max_{\mathbf{y}'} \sum_{i=1}^{n} \mathbf{w}^\top \mathbf{f}(y'_{i-1}, y'_i, \mathbf{x}, i)$$

Dynamic programming:
Define $\delta_i(y)$ = best score reaching position i with tag y

Recursion:

$$\delta_i(y) = \max_{y'} [\delta_{i-1}(y') + \mathbf{w}^\top \mathbf{f}(y', y, \mathbf{x}, i)]$$

Backpointers:

$$\psi_i(y) = \arg\max_{y'} [\delta_{i-1}(y') + \mathbf{w}^\top \mathbf{f}(y', y, \mathbf{x}, i)]$$

Viterbi Example: "The dog runs"

Sentence: "The dog runs"
Tagset: {DET, NOUN, VERB}

Learned weights (simplified):

Feature	Weight	Feature	Weight
⟨START, DET⟩	2.0	⟨DET, the⟩	3.0
⟨START, NOUN⟩	0.5	⟨NOUN, dog⟩	2.5
⟨DET, NOUN⟩	1.5	⟨VERB, runs⟩	2.0
⟨DET, VERB⟩	-0.5	⟨NOUN, VERB⟩	1.8
⟨NOUN, NOUN⟩	-1.0		

(Only showing subset of features for clarity)

Viterbi: Position 1 ("The")

Initialize: $\delta_1(y)$ for each tag y

For tag DET:

$$\begin{aligned}\delta_1(\text{DET}) &= w[\langle \text{START}, \text{DET} \rangle] + w[\langle \text{DET}, \text{the} \rangle] \\ &= 2.0 + 3.0 = 5.0\end{aligned}$$

For tag NOUN:

$$\begin{aligned}\delta_1(\text{NOUN}) &= w[\langle \text{START}, \text{NOUN} \rangle] + w[\langle \text{NOUN}, \text{the} \rangle] \\ &= 0.5 + 0.0 = 0.5 \quad \text{(assume emission weight} = 0\text{)}\end{aligned}$$

For tag VERB:

$$\delta_1(\text{VERB}) = 0.0 + 0.0 = 0.0$$

Best at position 1: DET with score 5.0

Viterbi: Position 2 ("dog")

For each possible tag at position 2, consider all previous tags:

For tag NOUN:

$$\begin{aligned}
\text{From DET: } & \delta_1(\text{DET}) + w[\langle \text{DET}, \text{NOUN} \rangle] + w[\langle \text{NOUN}, \text{dog} \rangle] \\
& = 5.0 + 1.5 + 2.5 = 9.0 \\
\text{From NOUN: } & \delta_1(\text{NOUN}) + w[\langle \text{NOUN}, \text{NOUN} \rangle] + w[\langle \text{NOUN}, \text{dog} \rangle] \\
& = 0.5 + (-1.0) + 2.5 = 2.0 \\
\text{From VERB: } & \delta_1(\text{VERB}) + \cdots = -1.0
\end{aligned}$$

$$\delta_2(\text{NOUN}) = \max\{9.0, 2.0, -1.0\} = 9.0$$

$$\psi_2(\text{NOUN}) = \text{DET}$$

Similarly compute $\delta_2(\text{VERB})$, $\delta_2(\text{DET})$...

Viterbi: Position 2 (continued)

For tag VERB at position 2:

$$\text{From DET: } 5.0 + (-0.5) + 0.0 = 4.5$$
$$\text{From NOUN: } 0.5 + 0.0 + 0.0 = 0.5$$

$$\delta_2(\text{VERB}) = 4.5, \quad \psi_2(\text{VERB}) = \text{DET}$$

After position 2:

Tag	δ_2	ψ_2
NOUN	9.0	DET
VERB	4.5	DET
DET	3.5	DET

Best path to "dog" as NOUN: START $\rightarrow$ DET $\rightarrow$ NOUN (score 9.0)

Viterbi: Position 3 ("runs")

For tag VERB at position 3:

$$\text{From NOUN: } 9.0 + 1.8 + 2.0 = 12.8$$
$$\text{From VERB: } 4.5 + 0.5 + 2.0 = 7.0$$
$$\text{From DET: } 3.5 + (-1.0) + 2.0 = 4.5$$

$$\delta_3(\text{VERB}) = 12.8, \quad \psi_3(\text{VERB}) = \text{NOUN}$$

Best final tag: VERB with score 12.8

Backtrack:

- Position 3: VERB (best ending)
- Position 2: $\psi_3(\text{VERB}) = \text{NOUN}$
- Position 1: $\psi_2(\text{NOUN}) = \text{DET}$

Final prediction: DET NOUN VERB

Viterbi: Complexity Analysis

Time complexity: $O(nK^2)$

- n positions in sequence
- K possible tags
- For each position and tag, consider K previous tags

Space complexity: $O(nK)$

- Store δ and ψ tables

Example:

- Sentence length: $n = 20$ words
- Penn Treebank: $K = 45$ tags
- Operations: $20 \times 45^2 = 40,500$ (very fast!)

Compare to naive enumeration: $45^{20} \approx 10^{33}$ sequences!

Handling Unknown Words

Problem: Test data contains words not seen in training

Solution 1: Morphological features

- Prefixes/suffixes: "ing" suggests VERB, "tion" suggests NOUN
- Capitalization: mid-sentence capitals often PROPN
- Contains-digit: often NUM
- Contains-hyphen: often ADJ

Solution 2: Word shape

- "Xxxx" (capitalized) $\rightarrow$ likely PROPN
- "xxxx" (lowercase) $\rightarrow$ various
- "XXXX" (all caps) $\rightarrow$ acronym, PROPN
- "dd/dd/dddd" $\rightarrow$ DATE

Key insight: These features generalize beyond the training vocabulary!

Example: Unknown Word Features

Unknown word: "unbelievable"

Even if never seen in training, features capture:

- $\langle y_i, \text{prefix=un} \rangle$ (negative prefix)
- $\langle y_i, \text{suffix=able} \rangle$ (adjective suffix)
- $\langle y_i, \text{suffix=ble} \rangle$
- $\langle y_i, \text{suffix=le} \rangle$
- $\langle y_i, \text{len} > 10 \rangle$

If training data had:

- "comfortable" / ADJ
- "reasonable" / ADJ
- "unbelievable" gets tagged as ADJ via suffix pattern!

This is why feature engineering matters!

Feature Engineering Best Practices

Good features are:

1. **Informative**: Correlate with correct tags
2. **Generalizable**: Work beyond training examples
3. **Sparse**: Most features $= 0$ for any input

Common patterns:

- Start simple (word + tag, bigram transitions)
- Add context (previous/next words)
- Add morphology (affixes, shape)
- Add conjunctions (word + previous tag)
- Test and iterate!

Overfitting risk:

- Too many rare features $\rightarrow$ memorize training data
- Solution: Feature count cutoffs (e.g., appears ≥ 5 times)

Named Entity Recognition (NER)

Task: Identify and classify named entities in text

Example:

Barack	Obama	visited	Microsoft	in	Seattle	.
B-PER	I-PER	O	B-ORG	O	B-LOC	O

Entity types:

- PER (person), ORG (organization), LOC (location), MISC (miscellaneous)

BIO tagging scheme:

- **B-X**: Beginning of entity type X
- **I-X**: Inside entity type X (continuation)
- **O**: Outside any entity

With 4 entity types: $4 \times 2 + 1 = 9$ tags total

Why BIO Tagging?

Question: Why not just use entity types (PER, ORG, LOC, O)?

Problem: Adjacent entities of same type

United	Airlines	beat	American
ORG	ORG	O	ORG

Without B/I distinction: Can't tell if "United Airlines" is one entity or two!

With BIO:

United	Airlines	beat	American
B-ORG	I-ORG	O	B-ORG

Clear: "United Airlines" is one ORG, "American" is another

B- tag marks entity boundaries

BIO Constraints

Valid transitions:

- O $\rightarrow$ O, O $\rightarrow$ B-X (for any X)
- B-X $\rightarrow$ I-X, B-X $\rightarrow$ O, B-X $\rightarrow$ B-Y (for any Y)
- I-X $\rightarrow$ I-X, I-X $\rightarrow$ O, I-X $\rightarrow$ B-Y (for any Y)

Invalid transitions:

- O $\rightarrow$ I-X (can't continue entity without starting it!)
- I-PER $\rightarrow$ I-ORG (can't change entity type mid-entity)

Enforcement:

- During Viterbi: Set score to $-\infty$ for invalid transitions
- Guarantees structurally valid output
- This is a key advantage of structured prediction!

Running Example for NER

Training sentence:

Tim	Cook	runs	Apple	in	California	.
B-PER	I-PER	O	B-ORG	O	B-LOC	O

Entity mentions:

- Person: "Tim Cook"
- Organization: "Apple"
- Location: "California"

We'll explore:

1. Features specific to NER
2. How capitalization and context matter
3. Multi-word entity handling
4. Training dynamics

Feature Templates for NER

Word-level features:

- $\langle y_i, x_i \rangle$, $\langle y_i, \text{lowercase}(x_i) \rangle$
- $\langle y_i, \text{is-capitalized}(x_i) \rangle$ *(crucial for NER!)*
- $\langle y_i, \text{is-all-caps}(x_i) \rangle$
- $\langle y_i, \text{word-shape}(x_i) \rangle$ (e.g., "Xxxx", "XX")

Affix features:

- $\langle y_i, \text{suffix}(x_i, k) \rangle$ for $k = 1, 2, 3$
- Helpful for organizations: "-Corp", "-Inc", "-Ltd"

Context features:

- $\langle y_i, x_{i-1}, x_{i+1} \rangle$ (both neighbors)
- $\langle y_i, x_{i-2}, x_{i-1} \rangle$ (left bigram context)

Gazetteers (optional):

- $\langle y_i, \text{in-person-list}(x_i) \rangle$
- $\langle y_i, \text{in-location-list}(x_i) \rangle$

Features for "Tim Cook runs Apple..."

Position 1: "Tim" / B-PER

- $\langle \text{START}, \text{B-PER} \rangle$ (transition)
- $\langle \text{B-PER}, \text{Tim} \rangle$ (emission)
- $\langle \text{B-PER}, \text{tim} \rangle$ (lowercase)
- $\langle \text{B-PER}, \text{capitalized=True} \rangle \leftarrow$ Key!
- $\langle \text{B-PER}, \text{shape=Xxx} \rangle$
- $\langle \text{B-PER}, \text{next=Cook} \rangle$ (right context)
- $\langle \text{B-PER}, \text{im} \rangle$ (suffix)

Why capitalization matters:

- Mid-sentence capital $\rightarrow$ likely entity
- "tim" (lowercase) would rarely be B-PER

Features for "Tim Cook runs Apple..." (cont.)

Position 2: "Cook" / I-PER

- $\langle$B-PER, I-PER$\rangle$ (transition from B-PER)
- $\langle$I-PER, Cook$\rangle$ (emission)
- $\langle$I-PER, capitalized=True$\rangle$
- $\langle$I-PER, prev=Tim$\rangle$ (left context)

Position 3: "runs" / O

- $\langle$I-PER, O$\rangle$ (transition ending entity)
- $\langle$O, runs$\rangle$ (emission)
- $\langle$O, lowercase=runs$\rangle$
- $\langle$O, capitalized=False$\rangle$ $\leftarrow$ Key!

Lowercase mid-sentence word $\rightarrow$ likely not an entity

Features for Multi-word Entity

Position 4: "Apple" / B-ORG

- $\langle \text{O}, \text{B-ORG} \rangle$ (transition)
- $\langle \text{B-ORG}, \text{Apple} \rangle$ (emission)
- $\langle \text{B-ORG}, \text{capitalized=True} \rangle$
- $\langle \text{B-ORG}, \text{prev=runs} \rangle$

Ambiguity: "Apple" could be:

- ORG (the company) $\leftarrow$ correct in this context
- O (the fruit)

Context helps:

- "runs Apple" suggests company (you can't "run" a fruit)
- Capitalized $\rightarrow$ more likely entity
- Features like $\langle \text{B-ORG}, \text{prev=runs} \rangle$ learn this!

Initial Training: Zero Weights

First training example:

```
Tim/B-PER Cook/I-PER runs/O Apple/B-ORG in/O California/B-LOC ./O
```

Initial model: $\mathbf{w} = \mathbf{0}$

Prediction with zero weights: All sequences score 0
Viterbi might predict (tie-breaking):

```
Tim/O Cook/O runs/O Apple/O in/O California/O ./O
```

(Predicts everything as O — common with random initialization!)

Update:

- Increase weights for all gold features
- Decrease weights for predicted features (mostly O transitions/emissions)

After First Update: Key Weights

Increased weights (sampled):

- $w[\langle \text{START}, \text{B-PER} \rangle] = +1$
- $w[\langle \text{B-PER}, \text{capitalized=True} \rangle] = +1$
- $w[\langle \text{B-PER}, \text{I-PER} \rangle] = +1$
- $w[\langle \text{I-PER}, \text{O} \rangle] = +1$
- $w[\langle \text{O}, \text{B-ORG} \rangle] = +1$
- $w[\langle \text{B-ORG}, \text{Apple} \rangle] = +1$
- $w[\langle \text{O}, \text{B-LOC} \rangle] = +1$

Decreased weights:

- $w[\langle \text{START}, \text{O} \rangle] = -1$
- $w[\langle \text{O}, \text{Tim} \rangle] = -1$
- $w[\langle \text{O}, \text{Apple} \rangle] = -1$

Model now prefers: Capitalized words at sentence start $\rightarrow$ B-PER

Second Training Example

Example 2:

```
Microsoft/B-ORG hired/O engineers/O in/O Seattle/B-LOC ./O
```

Current model has learned:

- Sentence-initial capitalized word $\rightarrow$ likely B-PER ($+1$)
- But "Microsoft" should be B-ORG!

Possible prediction:

```
Microsoft/B-PER hired/O engineers/O in/O Seattle/B-LOC ./O
```

(Gets "Microsoft" wrong)

Update effect:

- $w[\langle \text{START}, \text{B-ORG} \rangle] \leftarrow +1$
- $w[\langle \text{START}, \text{B-PER} \rangle] \leftarrow 0$ (was $+1$, now 0)
- $w[\langle \text{B-ORG}, \text{Microsoft} \rangle] \leftarrow +1$

Model learns: Some sentence-initial words are ORG, not PER!

Learning Entity Boundaries

Challenge: Multi-word entities

Example 3:

```
New/B-ORG York/I-ORG Times/I-ORG reported/O ...
```

What model learns:

- $w[\langle \text{B-ORG}, \text{I-ORG} \rangle]$ becomes positive
- $w[\langle \text{I-ORG}, \text{I-ORG} \rangle]$ becomes positive
- $w[\langle \text{I-ORG}, \text{capitalized=True} \rangle]$ positive

Effect:

- Consecutive capitalized words $\rightarrow$ likely single entity
- B-ORG followed by capitalized word $\rightarrow$ likely I-ORG
- I-ORG followed by lowercase $\rightarrow$ likely end of entity (transition to O)

Handling Ambiguity: "Apple"

Challenge: Same word, different labels in different contexts

Training examples:

1. "... runs **Apple/B-ORG** in ..."
2. "... ate an **apple/O** ..."

How does model distinguish?

Features capture context:

- $\langle$B-ORG, Apple, prev=runs$\rangle$ $\rightarrow$ positive weight
- $\langle$O, apple, prev=an$\rangle$ $\rightarrow$ positive weight
- $\langle$B-ORG, capitalized=True$\rangle$ $\rightarrow$ positive
- $\langle$O, capitalized=False$\rangle$ $\rightarrow$ positive

Capitalization + context words disambiguate!

Viterbi with BIO Constraints

Sentence: "Steve Jobs founded Apple"
Gold: B-PER I-PER O B-ORG

During Viterbi:
At position 2 ("Jobs"), consider transitions:

- From B-PER to I-PER: ✓Valid, compute score
- From B-PER to I-ORG: ✗ Invalid! Set score $= -\infty$
- From O to I-PER: ✗ Invalid! Set score $= -\infty$

Result:

- Viterbi automatically enforces BIO constraints
- Never produces invalid sequences
- No post-processing needed!

Detailed Viterbi for NER

Sentence: "Cook founded Apple"
Simplified weights:

Feature	Weight	Feature	Weight
⟨START, B-PER⟩	1.5	⟨B-PER, Cook⟩	2.0
⟨START, O⟩	0.5	⟨O, Cook⟩	0.3
⟨B-PER, O⟩	1.2	⟨O, founded⟩	1.0
⟨O, O⟩	0.8	⟨O, B-ORG⟩	1.5
⟨O, Apple⟩	-0.5	⟨B-ORG, Apple⟩	2.5

Position 1 ("Cook"):

$$\delta_1(\text{B-PER}) = 1.5 + 2.0 = 3.5$$
$$\delta_1(\text{O}) = 0.5 + 0.3 = 0.8$$

Detailed Viterbi (continued)

Position 2 ("founded"):
For tag O:

$$\text{From B-PER: } 3.5 + 1.2 + 1.0 = 5.7$$
$$\text{From O: } 0.8 + 0.8 + 1.0 = 2.6$$

$$\delta_2(\text{O}) = 5.7, \quad \psi_2(\text{O}) = \text{B-PER}$$

Position 3 ("Apple"):
For tag B-ORG:

$$\text{From O: } 5.7 + 1.5 + 2.5 = 9.7$$
$$\text{From B-PER: invalid transition from pos 2}$$

$$\delta_3(\text{B-ORG}) = 9.7$$

Common NER Errors

Error type 1: Boundary errors

- Gold: "New/B-LOC York/I-LOC"
- Pred: "New/O York/B-LOC"
- Wrong boundary, right type

Error type 2: Type errors

- Gold: "Apple/B-ORG"
- Pred: "Apple/B-LOC"
- Right boundaries, wrong type

Error type 3: Missed entities

- Gold: "Paris/B-LOC"
- Pred: "Paris/O"
- Common with rare entities

Evaluation: Exact match (boundaries + type must match)

- Precision: % of predicted entities that are correct

Improving NER Performance

Better features:

- Word embeddings (pre-trained)
- Character n-grams
- Brown clusters
- Gazetteers (lists of known entities)

More training data:

- Manually annotate more examples
- Use distant supervision (weak labels from KB)
- Data augmentation

Model improvements:

- Averaged perceptron (vs. standard)
- Higher-order features (trigrams)
- Semi-Markov models (segment-level features)

Averaged Perceptron Revisited

Problem: Final weights depend on example order

Solution: Average all intermediate weight vectors

Efficient implementation:

- Maintain cumulative sum $\mathbf{u}$
- After each update: $\mathbf{u} \leftarrow \mathbf{u} + \mathbf{w}$
- Track update count c
- Return $\mathbf{w}_{\text{avg}} = \mathbf{u}/c$

Impact on POS tagging:

- Standard perceptron: 95.5% accuracy
- Averaged perceptron: 97.0% accuracy
- Significant improvement with minimal cost!

Feature Conjunctions

Basic features: Individual properties

- $\langle y_i, x_i \rangle$
- $\langle y_i, \text{capitalized} \rangle$

Feature conjunctions: Combinations

- $\langle y_i, x_i, x_{i+1} \rangle$ (word + next word)
- $\langle y_i, x_i, \text{capitalized} \rangle$ (word + capitalization)
- $\langle y_{i-1}, y_i, x_i \rangle$ (tag bigram + word)

Example: "Apple" as ORG vs. O

- $\langle \text{B-ORG}, \text{Apple}, \text{prev=runs} \rangle$ captures: "When preceded by 'runs', 'Apple' is likely B-ORG"
- More specific than $\langle \text{B-ORG}, \text{Apple} \rangle$ alone

Trade-off: More expressive but more parameters (sparsity)

Learning Curves

How does accuracy improve with training?

Observations:

- Rapid initial improvement
- Training continues improving, dev plateaus overfitting risk
- Stop when dev accuracy plateaus (around epoch 7)

Error Analysis Example

Analyze mistakes to improve model

Sample errors from dev set:

1. "The us/B-LOC government..." (Gold: us/O)
   - Lowercase "us" misclassified as location (like "US")
   - Fix: Add feature $\langle y_i, \text{is-lowercase}(x_i) \rangle$
2. "...born in July/O..." (Gold: July/B-DATE)
   - Missing DATE entity type in our tagset
   - Fix: Add DATE category
3. "...CEO of Google/B-PER..." (Gold: Google/B-ORG)
   - Context "CEO of" strongly suggests ORG
   - Fix: Add feature $\langle y_i, \text{prev-bigram=CEO_of} \rangle$

Comparison: POS vs. NER

	POS Tagging	**NER**
Tags	45 (Penn Treebank)	9 (4 types $\times$ BIO + O)
Ambiguity	High (many words)	Lower (fewer entities)
Key features	Affixes, context	Capitalization, context
Boundary	Every word tagged	Multi-word entities
Difficulty	Moderate	Harder (rare entities)
Training size	Large corpora available	More limited

Common ground:

- Both are sequence labeling
- Both use Viterbi for inference
- Both benefit from rich features
- Same learning algorithm (structured perceptron)

Other Sequence Labeling Tasks

Same framework applies to:

Chunking: Identify syntactic phrases

```
[NP The cat] [VP sat] [PP on [NP the mat]]
```

Semantic Role Labeling: Identify predicate-argument structure

```
[A0 John] [V gave] [A1 a book] [A2 to Mary]
```

Super-sense tagging: Coarse semantic categories

```
dog.animal runs.motion
```

Chinese word segmentation: Identify word boundaries

```
/B /B /B /E (I love Beijing)
```

All use similar features, inference, and learning!

Implementation Sketch (Python)

```
class StructuredPerceptron:
    def __init__(self):
        self.weights = defaultdict(float)
        self.tagset = ['DET', 'NOUN', 'VERB', 'ADJ', ...]

    def extract_features(self, words, tags, i):
        """Extract features at position i"""
        features = []
        # Emission features
        features.append(('tag', tags[i], 'word', words[i]))
        features.append(('tag', tags[i], 'suffix', words[i][-2:]))
        # Transition features
        if i > 0:
            features.append(('trans', tags[i-1], tags[i]))
        # Context features
        if i > 0:
            features.append(('tag', tags[i], 'prev', words[i-1]))
        return features
```

Implementation (continued)

```
def score_sequence(self, words, tags):
    """Compute score for words with tags"""
    score = 0
    for i in range(len(words)):
        feats = self.extract_features(words, tags, i)
        for f in feats:
            score += self.weights[f]
    return score

def viterbi(self, words):
    """Find best tag sequence"""
    # Initialize DP table
    n, K = len(words), len(self.tagset)
    delta = [[float('-inf')] * K for _ in range(n)]
    psi = [[None] * K for _ in range(n)]

    # ... (Viterbi implementation)
```

Implementation (continued)

```
def train(self, data, epochs=10):
    """Train structured perceptron"""
    cumsum = defaultdict(float)  # For averaging
    updates = 0

    for epoch in range(epochs):
        random.shuffle(data)
        for words, gold_tags in data:
            # Predict
            pred_tags = self.viterbi(words)

            # Update if incorrect
            if pred_tags != gold_tags:
                # Extract features for both
                gold_feats = self.get_all_features(words, gold_tags)
                pred_feats = self.get_all_features(words, pred_tags)

                # Update weights
```

Practical Considerations

Feature hashing:

- Map feature strings to fixed-size array using hash function
- Trade-off: Collisions vs. memory efficiency
- Typical: Hash to $2^20 or 2^24 dimensions$

Sparse representations:

- Use dictionaries/hashmaps, not dense arrays
- Only store non-zero weights
- Critical for millions of features

Regularization:

- Feature count cutoffs: Remove features seen ¡ 5 times
- Weight clipping: Cap max absolute weight
- Early stopping: Stop when dev accuracy plateaus

Debugging Tips

Common issues:

1. **All predictions are O**:
   - Check feature extraction (are features firing?)
   - Verify Viterbi implementation
   - Ensure transition features work
2. **No improvement after training**:
   - Verify update rule (gold - pred, not pred - gold!)
   - Check if averaging is implemented
   - Ensure features are informative
3. **Memory issues**:
   - Too many features $\rightarrow$ Use feature pruning
   - Implement feature hashing
   - Use sparse representations

Key Takeaways

Structured Perceptron:

- Simple online learning algorithm
- Update: $\mathbf{w} \leftarrow \mathbf{w} + \Phi(\text{gold}) - \Phi(\text{pred})$
- Average weights for better performance!

Feature Engineering:

- POS: Morphology (affixes), context
- NER: Capitalization, gazetteers, context
- Conjunctions capture interactions

Viterbi Decoding:

- DP algorithm: $O(nK^2)$ instead of $O(K^n)$
- Can enforce hard constraints (BIO validity)

Applications:

- Many sequence labeling tasks use same framework

Structured Perceptron vs. Other Models

	Perceptron	CRF	MEMM	HMM
Training	Online	Batch	Batch	Counting
Normalization	None	Global	Local	Local
Features	Rich	Rich	Rich	Limited
Speed	Fast	Slow	Medium	Fast
Label bias	No	No	Yes	N/A
Probabilistic	No	Yes	Yes	Yes

When to use structured perceptron:

- Need fast training (online learning)
- Don't need calibrated probabilities
- Have good features
- Competitive accuracy with simpler implementation than CRF

Beyond Linear Models

Neural sequence models:

BiLSTM-CRF:

- Replace hand-crafted features with BiLSTM
- Keep CRF layer for transition scores
- State-of-the-art 2016-2018

Transformers:

- BERT + token classification layer
- Often no explicit structure modeling
- Current state-of-the-art

Why study structured perceptron?

- Understand structured prediction fundamentals
- Feature engineering intuitions carry over
- Fast, interpretable baseline
- Still used in resource-constrained settings

Exercise: Build Your Own Tagger

Suggested project:

1. Get data: CoNLL-2003 (NER) or Penn Treebank (POS)
2. Implement:
   - Feature extraction
   - Viterbi algorithm
   - Perceptron training loop
   - Evaluation (accuracy or F1)
3. Experiment:
   - Compare standard vs. averaged perceptron
   - Try different feature sets
   - Analyze errors
   - Add features to fix common mistakes
4. Compare to sklearn CRFsuite or existing implementations

Great way to deeply understand the algorithm!

Resources

Papers:

- Collins (2002): "Discriminative Training Methods for HMMs"
- Ratinov & Roth (2009): "Design Challenges and Misconceptions in NER"
- Tjong Kim Sang De Meulder (2003): "Introduction to CoNLL-2003 Shared Task"

Software:

- sklearn-crfsuite: Python CRF library
- Wapiti: Fast CRF/perceptron toolkit
- spaCy: Modern NLP library (neural models)

Datasets:

- Penn Treebank (POS): 45 tags, 1M tokens
- CoNLL-2003 (NER): English news, 4 entity types
- OntoNotes (multiple tasks): Large, diverse corpus